

# SuperKart - Product Sales Predictive system

An End-to-End ML Forecasting Platform – Containerized  
and Production-Ready

Uday Poluri | 21st April, 2026

# Contents / Agenda

- Executive Summary
- Business Problem Overview and Solution Approach
- EDA Results
- Data Preprocessing
- Model Building
- Model Performance Summary
- Deployment

# Executive Summary

- **Goal**

To build a reliable sales forecasting system that helps make better business decisions, deployable on any platform.

- **Methodology:**

- Evaluate and compare multiple forecasting models, fine-tune to identify the most accurate and reliable one.
- Deploy the model as a platform-independent service, accessible from anywhere.
- Validate the system by building a simple and intuitive interface.

# Executive Summary

- **Key Observations**

- Fruits & Vegetables and Snack foods are highest revenue-generating product categories indicating strong customer demand.
- Medium-sized stores lead in revenue generation, outperforming both Small and Large stores, suggesting an optimal store size sweet spot for sales performance.
- Tier 2 cities are our top revenue performers, likely because of lower competition compared to Tier 1 metros.
- Type 2 Supermarkets are the top revenue generators, while Food marts trail significantly, reinforcing the notion that larger store formats tend to drive higher sales volumes.
- Two models — Random Forest and XGBoost — performed well. Random Forest performed better and is our recommended choice, with room to improve further.
- The system is packaged into lightweight, standalone executable that can be deployed on any platform.

# Executive Summary

- **Recommendations**

- Deploy the Fine tuned Random Forest model in a containerized platform to ensure scalable and reliable performance.
- Conduct further model fine-tuning and evaluation to improve the forecast accuracy.
- Explore additional features such as seasonality, promotions, and regional economic indicators to improve accuracy
- Develop targeted sales strategies to boost the performance of low-volume product categories, including promotional incentives and bundling opportunities.
- Prioritize investment in store locations with higher customer base concentrations to maximize revenue potential.

# Business Problem Overview and Solution Approach

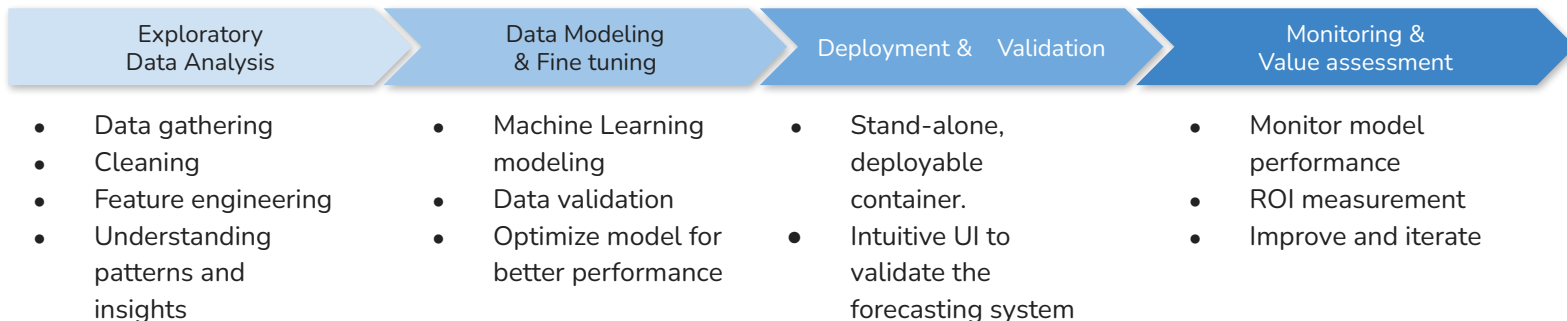
## Business Problem

SuperKart, a growing retail chain, needs a smarter way to plan its sales operations. A data-driven forecasting system — built on historical trends, industry patterns, and current market signals — will help accurately procure products, reduce inventory risks, cut down time spent on manual planning, and set reliable benchmarks for future growth

## Objective

To design, evaluate, and deploy a reliable sales forecasting system that enables smarter procurement, inventory planning, and business decision-making — accessible from any platform.

## Solution Approach



# Data Overview



## Data Source SuperKart.csv

Historical data, Industry trends, Sales pipelines

Size of the Data  
8,763 rows | 12 cols

- 16 Unique product types
- 4 Store types - Supermarket Type 1, Type 2, Food Mart, Departmental Stores
- 3 Store location types - Tier 1, Tier 2, Tier 3
- 3 Store sizes - Small, Medium and High

data.head(5)

|   | Product_Id | Product_Weight | Product_Sugar_Content | Product_Allocated_Area | Product_Type       | Product_MRP | Store_Id | Store_Establishment_Year | Store_Size | Store_Location_City_Type | Store_Type         | Product_Store_Sales_Total |
|---|------------|----------------|-----------------------|------------------------|--------------------|-------------|----------|--------------------------|------------|--------------------------|--------------------|---------------------------|
| 0 | FD6114     | 12.66          | Low Sugar             | 0.027                  | Frozen Foods       | 117.08      | OUT004   | 2009                     | Medium     | Tier 2                   | Supermarket Type2  | 2842.40                   |
| 1 | FD7839     | 16.54          | Low Sugar             | 0.144                  | Dairy              | 171.43      | OUT003   | 1999                     | Medium     | Tier 1                   | Departmental Store | 4830.02                   |
| 2 | FD5075     | 14.28          | Regular               | 0.031                  | Canned             | 162.08      | OUT001   | 1987                     | High       | Tier 2                   | Supermarket Type1  | 4130.16                   |
| 3 | FD8233     | 12.10          | Low Sugar             | 0.112                  | Baking Goods       | 186.31      | OUT001   | 1987                     | High       | Tier 2                   | Supermarket Type1  | 4132.18                   |
| 4 | NC1180     | 9.57           | No Sugar              | 0.010                  | Health and Hygiene | 123.67      | OUT002   | 1998                     | Small      | Tier 3                   | Food Mart          | 2279.36                   |

data.tail(5)

|      | Product_Id | Product_Weight | Product_Sugar_Content | Product_Allocated_Area | Product_Type       | Product_MRP | Store_Id | Store_Establishment_Year | Store_Size | Store_Location_City_Type | Store_Type        | Product_Store_Sales_Total |
|------|------------|----------------|-----------------------|------------------------|--------------------|-------------|----------|--------------------------|------------|--------------------------|-------------------|---------------------------|
| 8758 | NC7546     | 14.80          | No Sugar              | 0.016                  | Health and Hygiene | 140.53      | OUT004   | 2009                     | Medium     | Tier 2                   | Supermarket Type2 | 3806.5                    |
| 8759 | NC584      | 14.06          | No Sugar              | 0.142                  | Household          | 144.51      | OUT004   | 2009                     | Medium     | Tier 2                   | Supermarket Type2 | 5020.7                    |
| 8760 | NC2471     | 13.48          | No Sugar              | 0.017                  | Health and Hygiene | 88.58       | OUT001   | 1987                     | High       | Tier 2                   | Supermarket Type1 | 2443.4                    |
| 8761 | NC7187     | 13.89          | No Sugar              | 0.193                  | Household          | 168.44      | OUT001   | 1987                     | High       | Tier 2                   | Supermarket Type1 | 4171.8                    |
| 8762 | FD306      | 14.73          | Low Sugar             | 0.177                  | Snack Foods        | 224.93      | OUT002   | 1998                     | Small      | Tier 3                   | Food Mart         | 2186.0                    |

# Statistical Summary

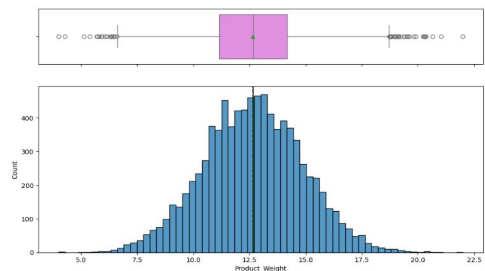
|                           | count  | unique | top                   | freq | mean        | std         | min    | 25%      | 50%     | 75%      | max    |
|---------------------------|--------|--------|-----------------------|------|-------------|-------------|--------|----------|---------|----------|--------|
| Product_Id                | 8763   | 8763   | FD306                 | 1    | NaN         | NaN         | NaN    | NaN      | NaN     | NaN      | NaN    |
| Product_Weight            | 8763.0 | NaN    | NaN                   | NaN  | 12.653792   | 2.21732     | 4.0    | 11.15    | 12.66   | 14.18    | 22.0   |
| Product_Sugar_Content     | 8763   | 4      | Low Sugar             | 4885 | NaN         | NaN         | NaN    | NaN      | NaN     | NaN      | NaN    |
| Product_Allocated_Area    | 8763.0 | NaN    | NaN                   | NaN  | 0.068786    | 0.048204    | 0.004  | 0.031    | 0.056   | 0.096    | 0.298  |
| Product_Type              | 8763   | 16     | Fruits and Vegetables | 1249 | NaN         | NaN         | NaN    | NaN      | NaN     | NaN      | NaN    |
| Product_MRP               | 8763.0 | NaN    | NaN                   | NaN  | 147.032539  | 30.69411    | 31.0   | 126.16   | 146.74  | 167.585  | 266.0  |
| Store_Id                  | 8763   | 4      | OUT004                | 4676 | NaN         | NaN         | NaN    | NaN      | NaN     | NaN      | NaN    |
| Store_Establishment_Year  | 8763.0 | NaN    | NaN                   | NaN  | 2002.032751 | 8.388381    | 1987.0 | 1998.0   | 2009.0  | 2009.0   | 2009.0 |
| Store_Size                | 8763   | 3      | Medium                | 6025 | NaN         | NaN         | NaN    | NaN      | NaN     | NaN      | NaN    |
| Store_Location_City_Type  | 8763   | 3      | Tier 2                | 6262 | NaN         | NaN         | NaN    | NaN      | NaN     | NaN      | NaN    |
| Store_Type                | 8763   | 4      | Supermarket Type2     | 4676 | NaN         | NaN         | NaN    | NaN      | NaN     | NaN      | NaN    |
| Product_Store_Sales_Total | 8763.0 | NaN    | NaN                   | NaN  | 3464.00364  | 1065.630494 | 33.0   | 2761.715 | 3452.34 | 4145.165 | 8000.0 |

## Observation:

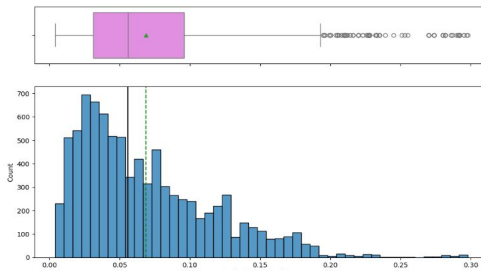
- Fruits and Vegetables are the top product categories, with most products having low sugar content.
- Product prices range from \$31 to \$266, with an average price of approximately \$147.
- The mean and median prices are closely aligned, suggesting a roughly normal price distribution with no significant skew.
- Most stores are medium-sized, with Supermarket Type 2 being the most common store format.
- Tier 2 cities have the highest concentration of stores across all city tiers.
- Store establishment years span from 1987 to 2009, representing over two decades of retail expansion.



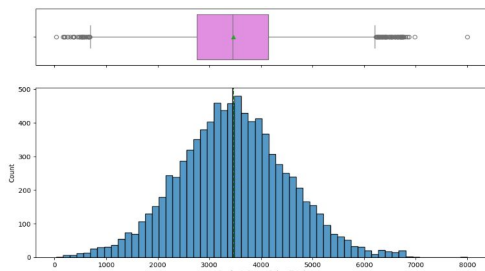
# EDA Results - Product Data Distribution



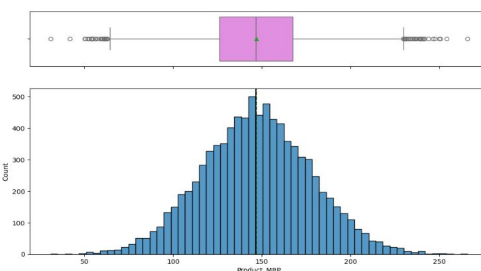
Product Weight



Product Area Allocated



Product Store Sales

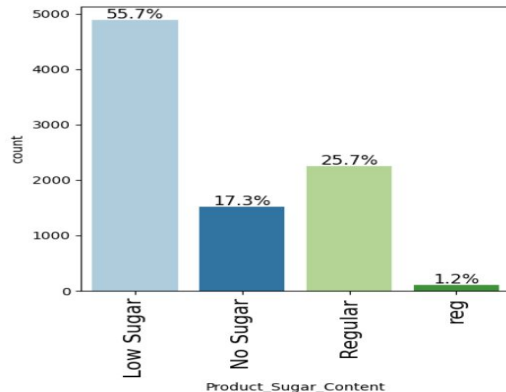


Product MRP

- **Product weight** - shows normal distribution with no significant skew, data has some outliers.
- **Product allocated area** - shows a right-skewed distribution, indicating that most products occupy a smaller area, with only a few products having significantly larger allocations — which is expected.
- **Product MRP** - shows a normally distributed pricing structure.
- **Product Store Sales** - shows a near-perfect bell shape, indicating a normally distributed sales pattern.

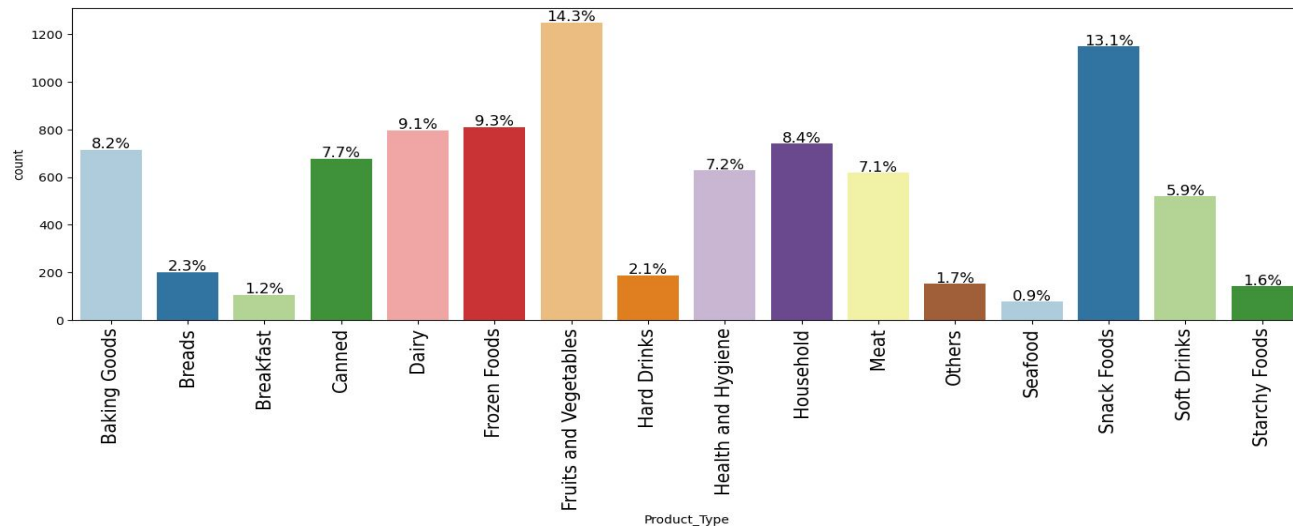
[Link to Appendix slide on data background check](#)

# EDA Results - Product Data Distribution



## Product Sugar Content

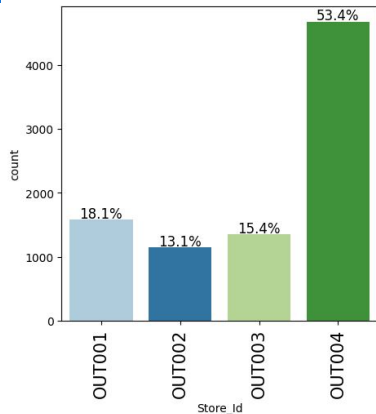
- Low Sugar dominates at 55.7%, making it the most common sugar content category by a wide margin.
- Regular comes in second at 25.7%, followed by No Sugar at 17.3%.
- The "reg" category accounts for only 1.2% — this looks like a data quality issue, likely a duplicate of "Regular" that needs to be cleaned up.



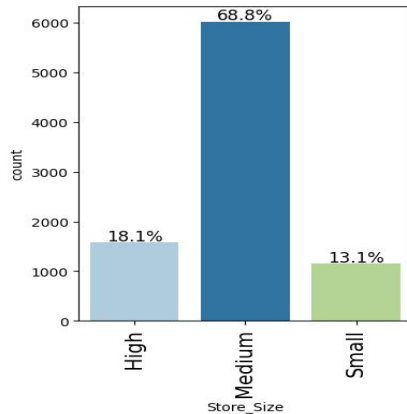
## Product Types

- Fruits and Vegetables is the most common product type at 14.3%, followed closely by Snack Foods at 13.1%.
- Frozen Foods (9.3%) and Dairy (9.1%) round out the top four categories.
- Seafood (0.9%), Starchy Foods (1.6%), and Others (1.7%) are the least represented categories.
- Overall, the distribution is fairly spread across categories, with no single type overwhelmingly dominating the dataset.

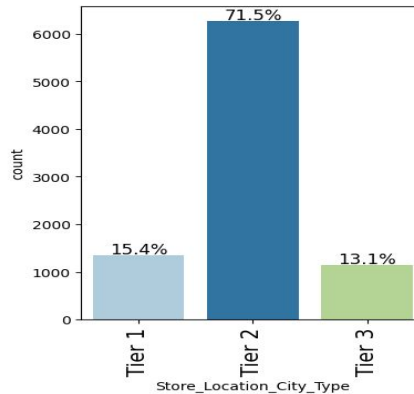
# EDA Results - Store Data Distribution



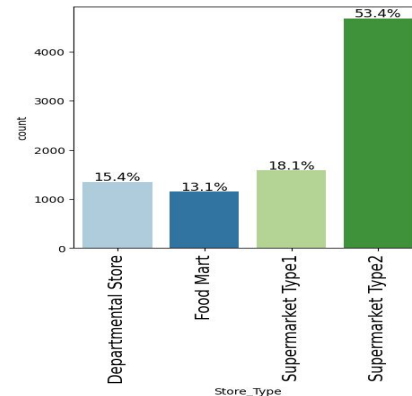
Store Id



Store Size



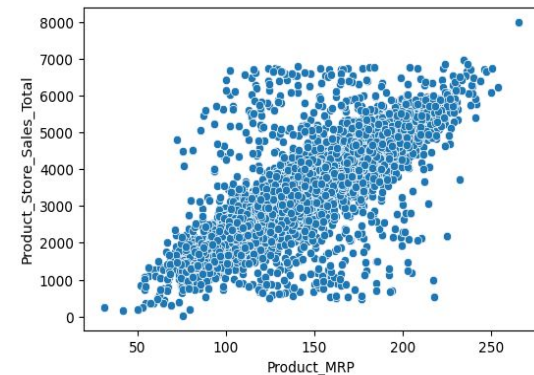
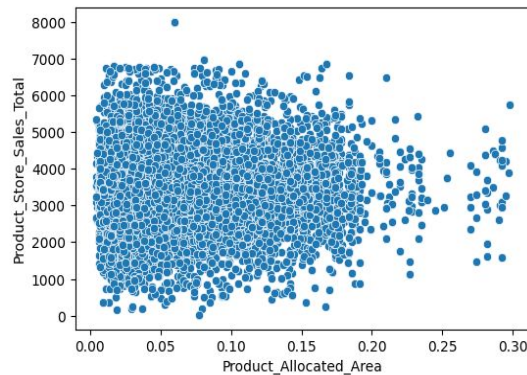
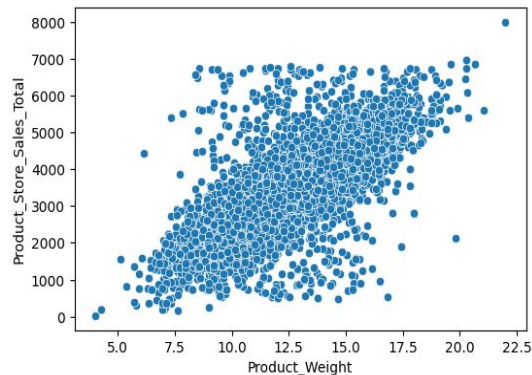
Store Location City Type



Store type

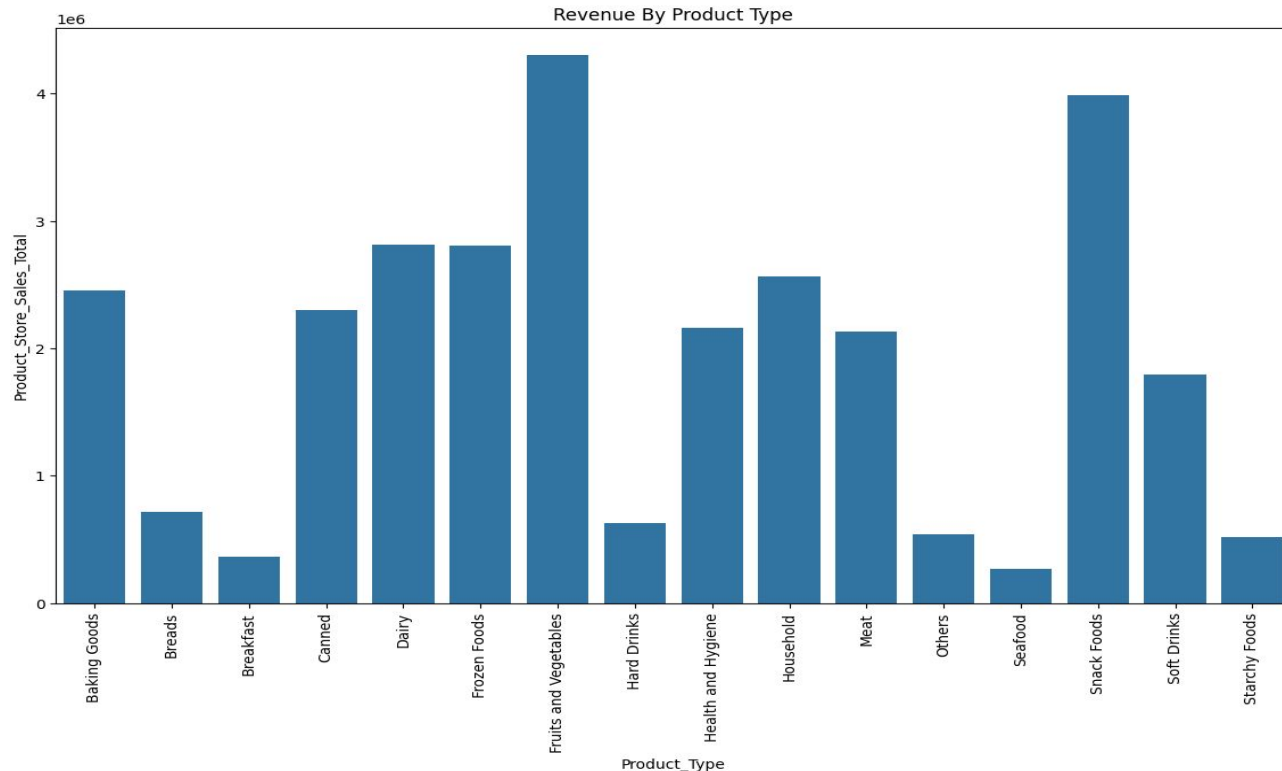
- OUT004 accounts for over half the dataset at 53.4%, while the remaining three stores — OUT001 (18.1%), OUT003 (15.4%), and OUT002 (13.1%) — are fairly evenly distributed.
- This heavy concentration in OUT004 introduces a store bias in the data, which could influence model predictions if store identity is a contributing factor.
- Medium-sized stores dominate the dataset at 69%, followed by High at 18.1% and Small at 13.1%, indicating an uneven representation across store sizes.
- Tier 2 cities overwhelmingly dominate the dataset at 71.5%, while Tier 1 (15.4%) and Tier 3 (13.1%) are comparably small and close to each other.
- Similar to store size, this heavy skew toward Tier 2 cities indicates an uneven representation across city types, which could introduce location bias in the model.
- Supermarket Type 2 dominates the dataset at 53%, followed by Supermarket Type 1 (18.1%), Departmental Stores (15.4%), and Food Marts (13.1%), showing a clear imbalance across store types.

# EDA Results - Product Store Sales vs Numeric features



- Store sales show a positive correlation with product weight, suggesting that heavier products tend to generate more sales.
- No clear correlation between Product Store sales and Product Allocated Area.
- Store sales tend to be higher for products with a greater MRP, pointing to a positive correlation between the two.
- Product MRP and Weight are likely to be strong predictors in the forecasting model.

# EDA - Product Store Sales by Product Types



- Fruits & Vegetables and Snack Foods are the top revenue generating product types.
- Mid-tier categories including Dairy, Frozen Foods, and Canned goods generate comparable, steady revenue.
- Seafood, Breakfast are least revenue generating Product types.
- These insights should help prioritize the inventory and promotions around top performing product categories, while exploring strategies for under performing.

# EDA Summary

- Dataset contains 8,763 records across 12 features, covering 16 product types, 4 store types, and 3 city tiers.
- Fruits & Vegetables and Snack Foods are the top revenue-generating categories, while Seafood and Breakfast are the least.
- Mid-tier categories like Dairy, Frozen Foods, and Canned Goods generate comparable, steady revenue.
- Product MRP and Product Weight show a positive correlation with sales — strong candidates as predictive features.
- Product Allocated Area shows no meaningful correlation with sales — likely a low-importance feature.
- Medium-sized stores and Tier 2 cities dominate the dataset, introducing potential bias — model performance may vary for underrepresented store sizes and locations.
- Supermarket Type 2 leads in total sales, with Food Marts being the least common store type.
- Product Store Sales data shows a near normal distribution, which is a good sign for ML modeling and reliability.

# Data Preprocessing

## Data Quality Checks

- **Duplicate Records:** No duplicate rows found in the training data.
- **Missing Values:** No missing values detected across features.
- **Outlier Detection:** Outliers were identified in all numerical features — Product Weight, Product Allocated Area, Product MRP, and Product Store Sales. Given the natural variability in retail data, these values are expected and no outlier treatment was applied.

## Feature Engineering:

- The Product\_Id column follows a pattern — two characters followed by a number. The first two characters were extracted into a new feature: Product\_ID\_Char.
- Three unique Product\_ID\_Chars were identified: FD (Food), NC (Non-Consumable), and DR (Drinks).
- Food comprises 11 product types, Drinks has 2, and Non-Consumables has 3.

# Data Preprocessing

## Feature Engineering (Continued):

- Products were further grouped into a new feature `Product_Type_Category` with two classes:
  - Perishable — Dairy, Meat, Fruits & Vegetables, Breakfast, Breads, Seafood
  - Non-Perishable — Frozen Foods, Canned, Baking Goods, Snack Foods, Starchy Foods, Hard D

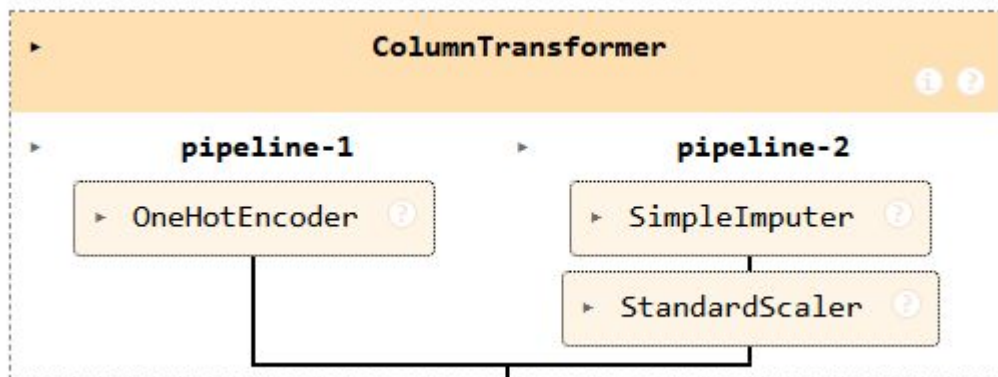
## Data Preparation for Modeling

- Features deemed non-predictive for regression — `Product_Id`, `Product_Type`, `Store_Id`, and `Store_Establishment_Year` — were dropped.
- Data was split into 75% training and 25% evaluation.
- All categorical features were encoded for model compatibility.



# Data Preprocessing

## Preprocessor



- A ColumnTransformer preprocessor is added to the ML pipeline to robustly handle unseen or unexpected data during inference.
- Pipeline-1: OneHotEncoder to ignore the unknown categories.
- Pipeline-2:
  - "Pipeline-2a: SimpleImputer replaces any missing numerical values with the median.
  - Pipeline-2b: StandardScaler normalizes the data to ensure consistent feature scaling.

# Model Evaluation Criteria

- The model forecasts Product Sales Store Total using historical data, industry trends, and current sales pipelines.
- The goal is to minimize prediction error in regression.
- Performance will be evaluated using RMSE, MAE, MAPE, R-Squared, and Adjusted R-Squared.
- RMSE and MAPE will serve as primary error metrics; Adjusted R-Squared will measure overall model fit, with a higher score indicating stronger explanatory power.

# Model Building - Approach

## Data & Pipeline Setup

- Two models are trained, evaluated, and compared — Random Forest and XGBoost Regressors
- Data is split into 75% training and 25% test/evaluation sets
- Model pipelines are constructed with two stages:
  - Preprocessor: One-Hot Encoding, Simple Imputer, and Standard Scaler
  - Estimator: The regressor model (Random Forest or XGBoost)

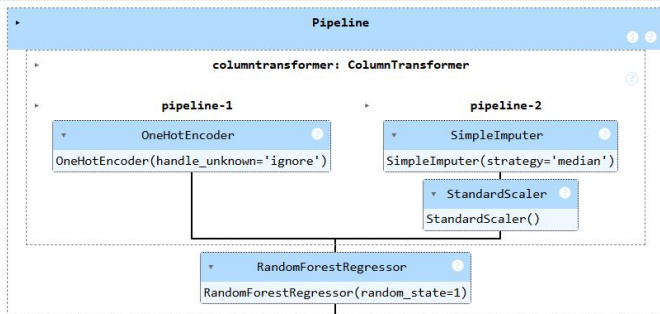
## Training & Optimization

- Baseline performance metrics are established using default model parameters as a reference point
- Hyperparameter tuning is performed to optimize model performance beyond the baseline

## Deployment

- The best-performing model is serialized (using joblib) for reuse without retraining
- REST APIs are developed to deploy the model on any platform for real-time predictions
- A Streamlit web application is developed as a lightweight UI to test and interact with the REST APIs

# Model 1: Random Forest Regressor



| Data     | RMSE   | MAE    | R <sup>2</sup> | Adj R <sup>2</sup> | MAPE  |
|----------|--------|--------|----------------|--------------------|-------|
| Training | 104.59 | 39.96  | 0.990          | 0.990              | 0.014 |
| Test     | 295.71 | 112.42 | 0.923          | 0.923              | 0.050 |

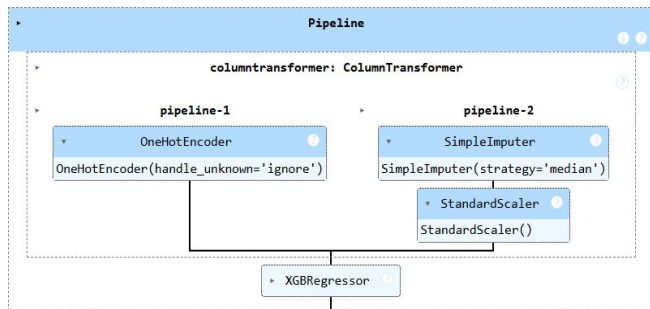
## Pipeline:

Preprocessor (One-Hot Encoding,  
Simple Imputer, Standard Scaler)  
→ RandomForestRegressor

## Observations:

- Model generalizes well to unseen data with a Test R<sup>2</sup> of 0.923
- A slight gap between training (R<sup>2</sup> = 0.990) and test performance suggests minor overfitting
- Hyperparameter tuning will be applied to further optimize the model
- Results establish a strong baseline ahead of hyperparameter tuning

# Model 2: XGBoost Regressor



| Data     | RMSE   | MAE    | R <sup>2</sup> | Adj R <sup>2</sup> | MAPE  |
|----------|--------|--------|----------------|--------------------|-------|
| Training | 118.62 | 58.09  | 0.987          | 0.987              | 0.019 |
| Test     | 309.44 | 138.03 | 0.916          | 0.916              | 0.056 |

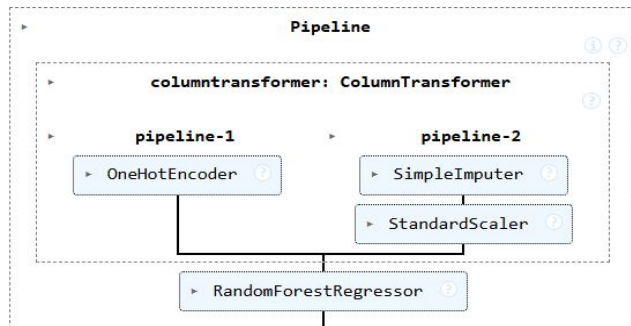
## Pipeline:

Preprocessor (One-Hot Encoding,  
Simple Imputer, Standard Scaler)  
→ XGBoostRegressor

## Observations:

- Model generalizes well to unseen data with a Test R<sup>2</sup> of 0.916
- A slight gap between training (R<sup>2</sup> = 0.987) and test performance suggests some overfitting
- Random Forest outperforms XGBoost across all baseline metrics
- Results establish a strong baseline ahead of hyperparameter tuning

# Model 1: Random Forest — Hyperparameter Tuned



| Data     | RMSE   | MAE    | R <sup>2</sup> | Adj R <sup>2</sup> | MAPE  |
|----------|--------|--------|----------------|--------------------|-------|
| Training | 196.95 | 79.56  | 0.965          | 0.965              | 0.027 |
| Test     | 288.93 | 114.47 | 0.927          | 0.926              | 0.051 |

## Observations:

- Hyperparameter tuning improved generalization — Test R<sup>2</sup> increased from 0.923 (baseline) to 0.927
- Training R<sup>2</sup> dropped from 0.990 to 0.965, indicating the model is less overfit and better generalized to unseen data
- The narrower training/test gap (R<sup>2</sup> difference: 0.039 vs 0.067 at baseline) confirms reduced overfitting
- XGBoost hyperparameter tuning will be performed next to determine the best overall model

## Pipeline:

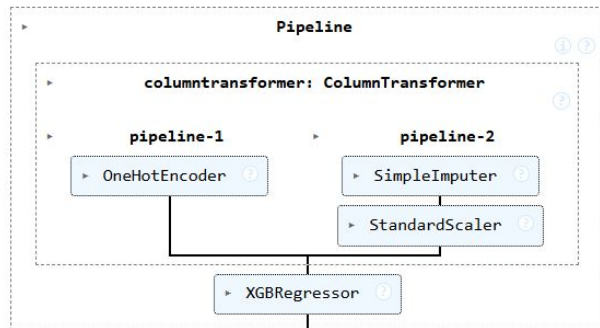
Hyperparameters -> RandomSearch  
Cross Validations

```

max_depth: [3, 5, 7, 10, 15, None ]
max_features :uniform(0.5, 0.5),
n_estimators: randint(75, 300)
min_samples_split: [2, 5, 10]

```

# Model 2: XGBoost — Hyperparameter Tuned



## Pipeline:

Hyperparameters -> Random Search  
Cross Validations

```

n_estimators: randint(75, 300)
subsample: uniform(0.5, 0.5)
gamma: uniform(0, 10)
colsample_bytree: uniform(0.3, 0.7)
colsample_bylevel: uniform(0.5, 0.5)
learning_rate: uniform(0.01, 0.3)
max_depth: randint(3, 10)
  
```

| Data     | RMSE   | MAE    | R <sup>2</sup> | Adj R <sup>2</sup> | MAPE  |
|----------|--------|--------|----------------|--------------------|-------|
| Training | 187.65 | 78.99  | 0.968          | 0.968              | 0.027 |
| Test     | 294.96 | 119.73 | 0.923          | 0.923              | 0.051 |

## Observations:

- Fine-tuned XGBoost performs better than its baseline.
- The training/test gap (R<sup>2</sup> difference: 0.045) reduced confirming less overfit relative to the baseline.
- Tuned XGBoost (R<sup>2</sup>=0.923) slightly underperforms tuned Random Forest (R<sup>2</sup>=0.927)
- There is room for further fine-tuning, particularly with an expanded parameter grid

# Model Performance Summary

| Model               | Data        | RMSE          | MAE           | R <sup>2</sup> | Adjusted R <sup>2</sup> | MAPE         |
|---------------------|-------------|---------------|---------------|----------------|-------------------------|--------------|
| RF Baseline         | Test        | 295.71        | 112.42        | 0.923          | 0.923                   | 0.050        |
| <b>RF Finetuned</b> | <b>Test</b> | <b>288.93</b> | <b>114.47</b> | <b>0.927</b>   | <b>0.926</b>            | <b>0.051</b> |
| XGB Baseline        | Test        | 309.44        | 138.03        | 0.916          | 0.916                   | 0.056        |
| XGB Finetuned       | Test        | 294.96        | 119.73        | 0.923          | 0.923                   | 0.051        |

- All four models demonstrate strong forecasting ability with Test R<sup>2</sup> scores above 0.916
- Hyperparameter tuning improved both models — confirming that RandomizedSearchCV was an effective optimization strategy.
- Random Forest Tuned is the best overall model with the highest Test R<sup>2</sup> (0.927) and lowest RMSE (288.93)
- XGBoost Tuned matches RF Baseline on R<sup>2</sup> (0.923) but falls slightly short of tuned RF across key metrics
- **Random Forest Tuned** is selected as the production model for sales forecasting deployment



# Model Serialization & Backend Deployment

## Model Serialization:

- Best performing model (Random Forest Fine Tuned,  $R^2=0.927$ ) is serialized using joblib
- Serialized model is saved as a .joblib file for reuse without retraining
- Model can be loaded and served instantly in any Python or Deployment environment

## REST API Development (Backend):

- REST APIs are built using the Flask web framework
- The API is deployed on HuggingFace Spaces, accessible at:
  - **Space:** [SuperKart Forecast API Space](#)
  - **Prediction Endpoint:** [Prediction Endpoint /v1/predict](#) — accepts input data and returns sales forecast
- A lightweight Docker image is created to deploy the API as a containerized application on any platform (AWS, GCP, Azure, or local server)

# Frontend - Streamlit Application

## Streamlit Web Application:

- A lightweight web application is developed using Streamlit and deployed on HuggingFace Spaces
- Accessible at: <https://huggingface.co/spaces/udx1/superkart-sales-forecast>
- Connects directly to the Flask REST API prediction endpoint for real-time forecasting

## User Input:

- Product details: Product Weight, Sugar Content, Allocated Area, MRP, Product ID, Product Type Category
- Store details: Store Size, Location City Type, Store Type, Store Age (Years)

## How It Works:

- User fills in the product and store details via dropdown menus and numeric inputs
- Clicks the "Predict" button to call the REST API
- App instantly displays: "Predicted Product Store Sales Total: \$XX.XX"

# Summary

- Two regression models — Random Forest and XGBoost — were built, evaluated, and compared to forecast SuperKart Product Sales Store Total
- All four models demonstrated strong forecasting ability with Test  $R^2$  scores above 0.916
- RandomizedSearchCV hyperparameter tuning improved both models — confirming it as an effective optimization strategy over GridSearchCV
- Random Forest Tuned is the best overall model with a Test  $R^2$  of 0.927, RMSE of 288.93, and MAPE of 5.1% — indicating strong real-world forecasting accuracy
- The model is successfully serialized, containerized via Docker, and deployed as a REST API on HuggingFace Spaces — accessible from any platform
- A Streamlit web application provides a user-friendly interface for real-time sales predictions without any coding

# Recommendations

- Further hyperparameter tuning with an expanded parameter grid could improve performance.
- Explore additional features such as seasonality, promotions, and regional economic indicators to improve accuracy
- Consider other models and ensemble approaches — combining Random Forest and XGBoost predictions may yield better results
- Implement automated retraining pipelines as new sales data becomes available to keep the model current
- Extend the Streamlit UI to support batch predictions via CSV upload for business users managing multiple stores

# Appendix

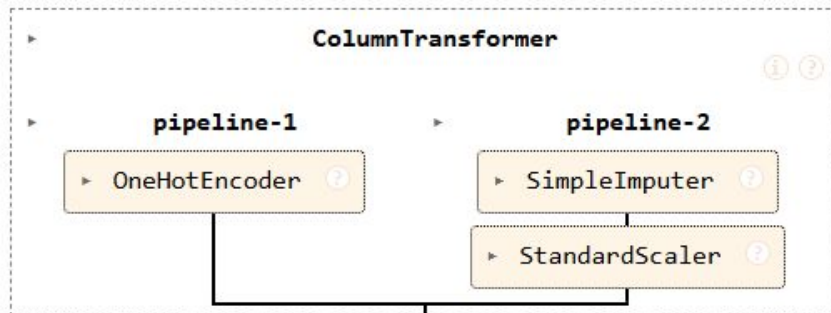
# Pipeline Setup

# Create a preprocessing pipeline for the categorical features

```
from sklearn.impute import SimpleImputer
```

```
preprocessor = make_column_transformer(
    (Pipeline(['encoder', OneHotEncoder(handle_unknown='ignore')]), categorical_features),
    (Pipeline(['num_imputer', SimpleImputer(strategy='median')], ('scaler', StandardScaler()))), numerical_features)
)
```

preprocessor



# Random Forest Model - Baseline

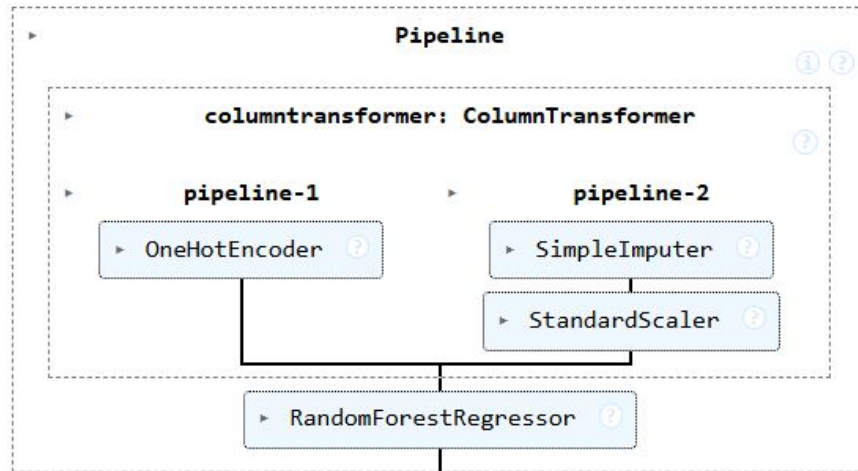
## Random Forest Model

[ ]

```
# Random forest regressor

rf_estimator = RandomForestRegressor(random_state=1)
rf_estimator = make_pipeline(preprocessor, rf_estimator)
rf_estimator.fit(X_train, y_train)
```

▼

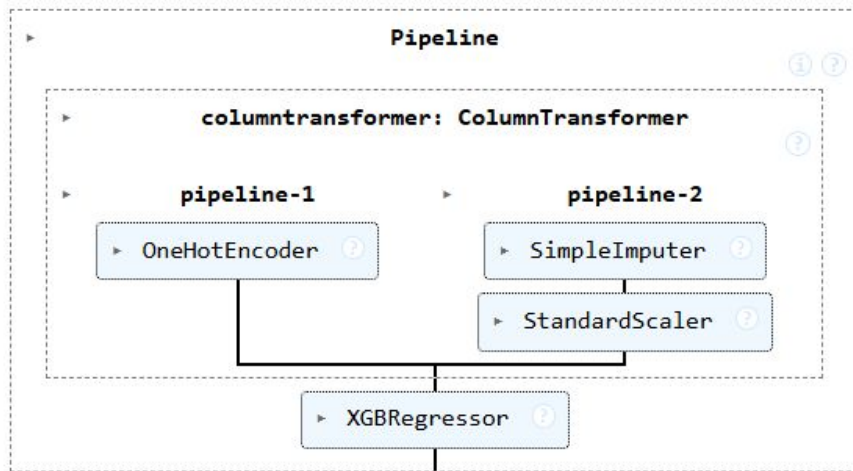


# XGBoost Regressor - Baseline

## XGBoost Regressor

```
# xgboost regressor
```

```
xgb_estimator = XGBRegressor(random_state=1)
xgb_estimator = make_pipeline(preprocessor,xgb_estimator)
xgb_estimator.fit(X_train, y_train)
```





# Random Forest - Fine Tuning

```

▶ from sklearn.model_selection import RandomizedSearchCV
  from scipy.stats import randint, uniform

  rf_tuned = RandomForestRegressor(random_state=1)
  rf_tuned = make_pipeline(preprocessor, rf_tuned)

  # Parameter distributions instead of fixed grid
  parameters = {
      "randomforestregressor__max_depth": [3, 5, 7, 10, 15, None],
      "randomforestregressor__max_features": uniform(0.5, 0.5), # samples between 0.5-1.0
      "randomforestregressor__n_estimators": randint(75, 300),
      "randomforestregressor__min_samples_split": randint(2, 10),
  }

  # RandomizedSearchCV - replacement for GridSearchCV
  grid_obj = RandomizedSearchCV(
      rf_tuned,
      parameters,
      n_iter=50,
      scoring='r2',
      cv=5,
      n_jobs=-1,
      random_state=42,
      verbose=2
  )
  grid_obj = grid_obj.fit(X_train, y_train)

  # Set to best combination of parameters
  rf_tuned = grid_obj.best_estimator_

  # Fit the best algorithm to the data
  rf_tuned.fit(X_train, y_train)

```

# XGBoost - Fine Tuning

```
xgb_tuned = XGBRegressor(random_state=1)
xgb_tuned = make_pipeline(preprocessor, xgb_tuned)

# Parameter distributions instead of fixed grid
parameters = {
    "xgbregressor__n_estimators": randint(75, 300),
    "xgbregressor__subsample": uniform(0.5, 0.5),      # samples between 0.5-1.0
    "xgbregressor__gamma": uniform(0, 10),             # samples between 0-10
    "xgbregressor__colsample_bytree": uniform(0.3, 0.7), # samples between 0.3-1.0
    "xgbregressor__colsample_bylevel": uniform(0.5, 0.5), # samples between 0.5-1.0
    "xgbregressor__learning_rate": uniform(0.01, 0.3), # bonus param - helps a lot!
    "xgbregressor__max_depth": randint(3, 10),         # bonus param - controls overfitting
}

# RandomizedSearchCV
grid_obj = RandomizedSearchCV(
    xgb_tuned,
    parameters,
    n_iter=50,
    scoring='r2',
    cv=5,
    n_jobs=-1,
    random_state=42,    # reproducibility
    verbose=2           # track progress
)
grid_obj = grid_obj.fit(X_train, y_train)

# Set to best combination of parameters
xgb_tuned = grid_obj.best_estimator_

# Fit the best algorithm to the data
```

# Model Performance - Comparision

... Training performance comparison:

|                | Random Forest Regressor | Random Forest Regressor - Tuned | XGBoost Regressor | XGBoost Regressor - Tuned |
|----------------|-------------------------|---------------------------------|-------------------|---------------------------|
| RMSE           | 104.588141              | 196.951309                      | 118.628943        | 187.656987                |
| MAE            | 39.957071               | 79.565829                       | 58.099567         | 78.993546                 |
| R-squared      | 0.990333                | 0.965718                        | 0.987563          | 0.968877                  |
| Adj. R-squared | 0.990317                | 0.965662                        | 0.987542          | 0.968826                  |
| MAPE           | 0.014119                | 0.027084                        | 0.019789          | 0.027430                  |



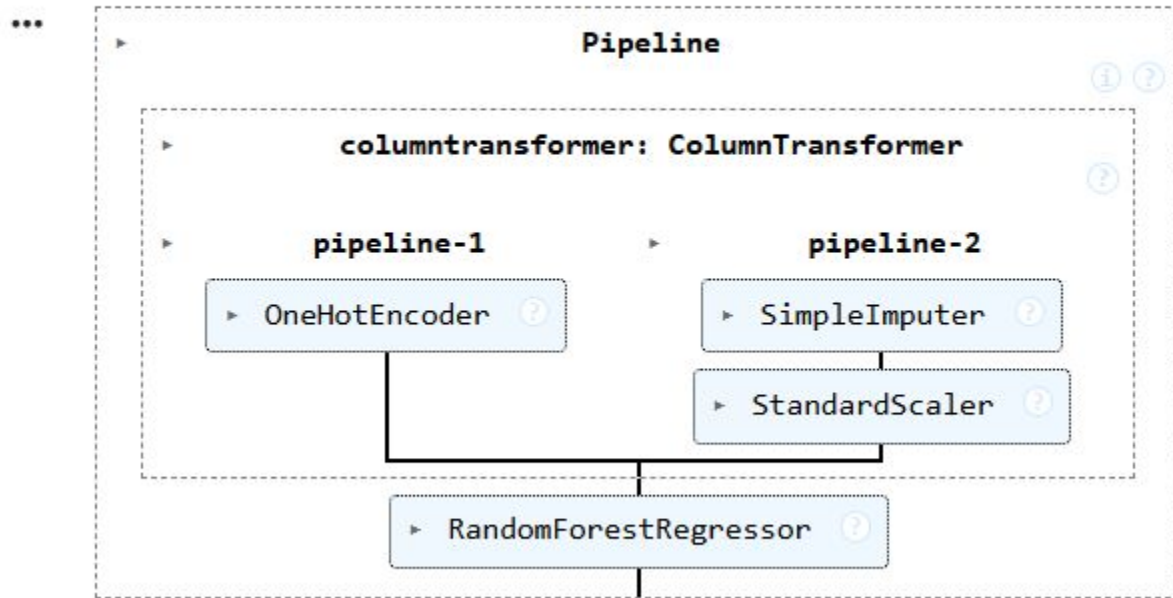
Test performance comparison:

|                | Random Forest Regressor | Random Forest Regressor - Tuned | XGBoost Regressor | XGBoost Regressor - Tuned |
|----------------|-------------------------|---------------------------------|-------------------|---------------------------|
| RMSE           | 295.708332              | 288.939748                      | 309.448421        | 294.965026                |
| MAE            | 112.423789              | 114.477750                      | 138.032740        | 119.734035                |
| R-squared      | 0.923587                | 0.927045                        | 0.916321          | 0.923971                  |
| Adj. R-squared | 0.923295                | 0.926766                        | 0.916001          | 0.923680                  |
| MAPE           | 0.050145                | 0.051438                        | 0.056671          | 0.051830                  |



# Best Model - Random Forest Finetuned

```
▶ best_model = rf_tuned  
best_model
```



# Model Serialization

## Model Serilization

```
1 # Create a folder for storing the deployment files.  
os.makedirs("backend_files", exist_ok=True)
```

```
1 # Define the file path to save (serialize) the trained model along with the data preprocessing steps  
saved_model_path = "backend_files/SuperKart_Model_v1.joblib"
```

```
1 # Save the best trained model pipeline using joblib  
joblib.dump(best_model, saved_model_path)  
  
print(f"Model saved successfully at {saved_model_path}")
```

```
Model saved successfully at backend_files/SuperKart_Model_v1.joblib
```

```
1 # Load the saved model pipeline from the file  
saved_model = joblib.load("backend_files/SuperKart_Model_v1.joblib")  
  
# Confirm the model is loaded  
print("Model loaded successfully.")
```

```
Model loaded successfully.
```

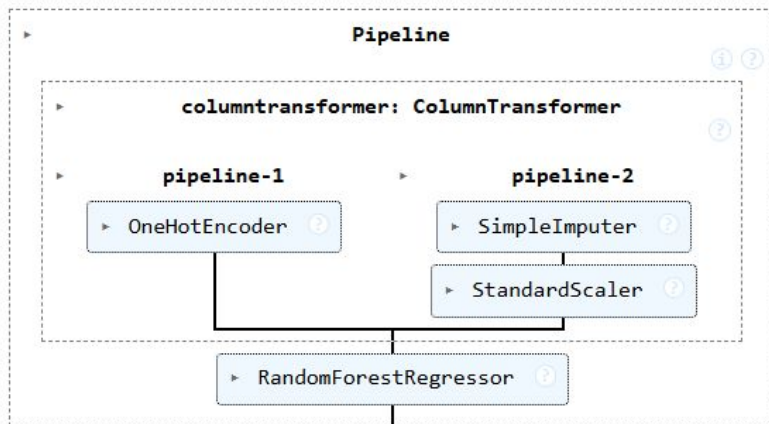
# Model Serialization - Reload Saved model

```
# Load the saved model pipeline from the file
saved_model = joblib.load("backend_files/SuperKart_Model_v1.joblib")

# Confirm the model is loaded
print("Model loaded successfully.")
```

... Model loaded successfully.

saved\_model



```
saved_model.predict(X_test)
```

```
array([3709.72534256, 5853.6653369 , 4456.48756815, ..., 3100.00114172,
       3172.56507711, 3554.83223897])
```

- As we can see, the model can be directly used for making predictions without any retraining.

# Deployment - Backend

## Flask Web Framework

```
%%writefile backend_files/app.py

# Import necessary libraries
import numpy as np
import joblib # For loading the serialized model
import pandas as pd # For data manipulation
from flask import Flask, request, jsonify # For creating the Flask API

# Initialize Flask app with a name
superkart_api = Flask("SuperKart")

# Load the trained churn prediction model
model = joblib.load("SuperKart_Model_v1.joblib")

# Define a route for the home page
@superkart_api.get('/')
def home():
    return "Welcome to SuperKart API - A place to do some kool predictions :)"

# Define an endpoint to predict churn for a single customer
@superkart_api.post('/v1/predict')
def predict_sales():
    # Get JSON data from the request
    data = request.get_json()
```

```
# Get JSON data from the request
data = request.get_json()

# Extract relevant customer features from the input data. The order of the column names matters.
sample = {
    'Product_Weight': data['Product_Weight'],
    'Product_Sugar_Content': data['Product_Sugar_Content'],
    'Product_Allocated_Area': data['Product_Allocated_Area'],
    'Product_MRP': data['Product_MRP'],
    'Store_Size': data['Store_Size'],
    'Store_Location_City_Type': data['Store_Location_City_Type'],
    'Store_Type': data['Store_Type'],
    'Product_Id_char': data['Product_Id_char'],
    'Store_Age_Years': data['Store_Age_Years'],
    'Product_Type_Category': data['Product_Type_Category']
}

# Convert the extracted data into a DataFrame
input_data = pd.DataFrame([sample])

# Make a churn prediction using the trained model
prediction = model.predict(input_data).tolist()[0]

# Return the prediction as a JSON response
return jsonify({'Sales': prediction})

# Run the Flask app in debug mode
if __name__ == '__main__':
    superkart_api.run(debug=True)
```

... Writing backend\_files/app.py



# Deployment - Backend

## Dependencies File

```
%%writefile backend_files/requirements.txt
pandas==2.2.2
numpy==2.0.2
scikit-learn==1.6.1
seaborn==0.13.2
joblib==1.4.2
xgboost==2.1.4
joblib==1.4.2
Werkzeug==2.2.2
flask==2.2.2
gunicorn==20.1.0
requests==2.32.3
uvicorn[standard]
streamlit==1.43.2
```

Writing backend\_files/requirements.txt

## Dockerfile

```
%%writefile backend_files/Dockerfile
FROM python:3.9-slim

# Set the working directory inside the container
WORKDIR /app #Complete the code to mention the command in Docker to set the working directory

# Copy all files from the current directory to the container's working directory
COPY . .

# Install dependencies from the requirements file without using cache to reduce image size
RUN pip install --no-cache-dir --upgrade -r requirements.txt

# Define the command to start the application using Gunicorn with 4 worker processes
# - '-w 4': Uses 4 worker processes for handling requests
# - '-b 0.0.0.0:7860': Binds the server to port 7860 on all network interfaces
# - 'app:app': Runs the Flask app (assuming 'app.py' contains the Flask instance named 'app')
CMD ["gunicorn", "-w", "4", "-b", "0.0.0.0:7860", "app:superkart_api"]
```

Writing backend\_files/Dockerfile



# Deployment - Setting up a Hugging Face Docker Space for the Backend

```
# Import the login function from the huggingface_hub library
from huggingface_hub import login
from google.colab import userdata
token = userdata.get('SK_HF_TOKEN')

# Login to your Hugging Face account using your access token
# Replace "YOUR_HUGGINGFACE_TOKEN" with your actual token
#login(token="YOUR_HUGGINGFACE_TOKEN") # You can get your token from https://huggingface.co/settings/tokens
login(token=token) #Complete the code to define the access token

# Import the create_repo function from the huggingface_hub library
from huggingface_hub import create_repo
```



```
# Try to create the repository for the Hugging Face Space
try:
    create_repo("superkart-sales-forecast-api",
                repo_type="space", # Specify the repository type as "space"
                space_sdk="docker", # Specify the space SDK as "docker"
                private=False # Set to True if you want the space to be private
    )
except Exception as e:
    # Handle potential errors during repository creation
    if "RepositoryAlreadyExistsError" in str(e):
        print("Repository already exists. Skipping creation.")
```

# Deployment - Uploading Files to Hugging Face Space (Docker Space)

+ Code

+ Text

```
# for hugging face space authentication to upload files

access_key = token
repo_id = "udx1/superkart-sales-forecast-api" # repo id.

# Login to Hugging Face platform with the access token
login(token=access_key)

# Initialize the API
api = HfApi()

# Upload Streamlit app files stored in the folder called deployment_files
api.upload_folder(
    folder_path="backend_files",
    repo_id=repo_id, # Hugging face space id
    repo_type="space", # Hugging face repo type "space"
)
```

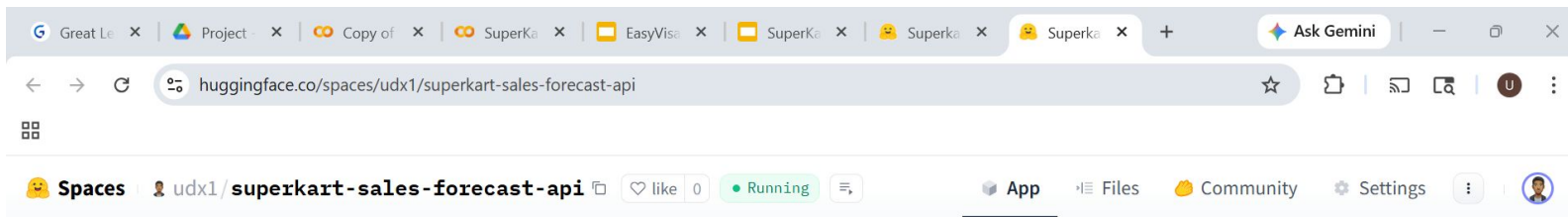
Processing Files (1 / 1) : 100%  8.23MB / 8.23MB, 6.86MB/s

New Data Upload : 100%  8.23MB / 8.23MB, 6.86MB/s

...SuperKart\_Model\_v1.joblib: 100%  8.23MB / 8.23MB

```
CommitInfo(commit_url='https://huggingface.co/spaces/udx1/superkart-sales-forecast-api/commit/33b6540fa3a747d672ee34a5c1e386f39b954747',
commit_message='Upload folder using huggingface_hub', commit_description='', oid='33b6540fa3a747d672ee34a5c1e386f39b954747', pr_url=None,
repo_url=RepoUrl('https://huggingface.co/spaces/udx1/superkart-sales-forecast-api', endpoint='https://huggingface.co', repo_type='space',
repo_id='udx1/superkart-sales-forecast-api'), pr_revision=None, pr_num=None)
```

# API - HuggingFace Space



# Deployment - Front End

## Streamlit for Interactive UI



```
# Create a folder for storing the files needed for frontend UI deployment
os.makedirs("frontend_files", exist_ok=True)
```

```
%%writefile frontend_files/app.py
```

```
import streamlit as st
import requests
```

```
st.title("SuperKart - Product Sales Predictions.") #Complete the code to define the title of the app.
```

```
# Input fields for product and store data
```

```
Product_Weight = st.number_input("Product Weight", min_value=0.0, value=12.66)
```

```
Product_Sugar_Content = st.selectbox("Product Sugar Content", ["Low Sugar", "Regular", "No Sugar"])
```

```
Product_Allocated_Area = st.number_input("Product Allocated Area", min_value=0.0)
```

```
Product_MRP = st.number_input("Product MRP", min_value=0.0)
```

```
Store_Size = st.selectbox("Store Size", ["Small", "Medium", "High"])
```

```
Store_Location_City_Type = st.selectbox("Store Location City Type", ['Tier 1', 'Tier 2', 'Tier 3'])
```

```
Store_Type = st.selectbox("Store Type", ['Departmental Store', 'Food Mart', 'Supermarket Type1', 'Supermarket Type2'])
```

```
Product_Id_char = st.selectbox("Product ID Char", ['DR', 'FD', 'NC'])
```

```
Store_Age_Years = st.number_input("Store Age Years", min_value=0.0)
```

```
Product_Type_Category = st.selectbox("Product Type Category", ['Non Perishables', 'Perishables'])
```

# Deployment - Front End

```
product_data = {
    "Product_Weight": Product_Weight,
    "Product_Sugar_Content": Product_Sugar_Content,
    "Product_Allocated_Area": Product_Allocated_Area,
    "Product_MRP": Product_MRP,
    "Store_Size": Store_Size,
    "Store_Location_City_Type": Store_Location_City_Type,
    "Store_Type": Store_Type,
    "Product_Id_char": Product_Id_char,
    "Store_Age_Years": Store_Age_Years,
    "Product_Type_Category": Product_Type_Category
}

if st.button("Predict", type='primary'):
    try:
        response = requests.post("https://udx1-superkart-sales-forecast-api.hf.space/v1/predict", json=product_data)
        if response.status_code == 200:
            result = response.json()
            predicted_sales = result["Sales"]
            st.write(f"Predicted Product Store Sales Total: ${predicted_sales:.2f}")
        else:
            st.error(f"Error {response.status_code}: {response.text}")
    except Exception as e:
        st.error(f"Something went wrong: {str(e)}")
```

... Writing frontend\_files/app.py

# Deployment - Front End

## Dependencies File

```
%%writefile frontend_files/requirements.txt
requests==2.32.3
streamlit==1.45.0
```

Writing frontend\_files/requirements.txt

## Dockerfile

```
%%writefile frontend_files/Dockerfile
# Use a minimal base image with Python 3.9 installed
FROM python:3.9-slim

# Set the working directory inside the container to /app
WORKDIR /app

# Copy all files from the current directory on the host to the container's /app directory
COPY . .

# Install Python dependencies listed in requirements.txt
RUN pip3 install -r requirements.txt

# Define the command to run the Streamlit app on port 8501 and make it accessible externally
CMD ["streamlit", "run", "app.py", "--server.port=8501", "--server.address=0.0.0.0", "--server.enableXsrfProtection=false"]

# NOTE: Disable XSRF protection for easier external access in order to make batch predictions
```



# Deployment - Front End

## Uploading Files to Hugging Face Space (Streamlit Space)

```
access_key = token
repo_id = "udx1/superkart-sales-forecast"

# Login to Hugging Face platform with the access token
login(token=access_key)

# Initialize the API
api = HfApi()

# Upload Streamlit app files stored in the folder called deployment_files
api.upload_folder(
    folder_path="frontend_files",
    repo_id=repo_id, # Hugging face space id
    repo_type="space", # Hugging face repo type "space"
)
```

```
No files have been modified since last commit. Skipping to prevent empty commit.
WARNING:huggingface_hub.hf_api:No files have been modified since last commit. Skipping to prevent empty commit.
CommitInfo(commit_url='https://huggingface.co/spaces/udx1/superkart-sales-forecast/commit/6511673795e2c9420eaf5722021fbb35995569f3',
commit_message='Upload folder using huggingface_hub', commit_description='', oid='6511673795e2c9420eaf5722021fbb35995569f3', pr_url=None,
repo_url=RepoUrl('https://huggingface.co/spaces/udx1/superkart-sales-forecast', endpoint='https://huggingface.co', repo_type='space',
repo_id='udx1/superkart-sales-forecast'), pr_revision=None, pr_num=None)
```

# Streamlit Application

Great Learning x Project x Copy of x SuperKart x EasyVisio x SuperKart x SuperKart x SuperKart x Ask Gemini

huggingface.co/spaces/udx1/superkart-sales-forecast

## SuperKart - Product Sales Predictions.

Product Weight  
12.66

Product Sugar Content  
Low Sugar

Product Allocated Area  
0.00

Product MRP  
0.00

Store Size  
Small

Store Location City Type  
Tier 1

Store Type  
Departmental Store

Product ID Char  
DR

Store Age Years  
0.00

Product Type Category  
Non Perishables

Predict





**Happy Learning !**

